Project Explanation

LPC1768-Based Two-Lane Racing Game with Analog Joystick Steering

ICT 3144 Embedded Systems Lab Mini Project, MIT Manipal

Tanisha Saha (240911398) | Avaneeth Gowda L (240911368) | Powel Lawrence Lewis (240911458)

---

1. The Project in Brief

We built a two-lane racing game that runs on the **ALS-SDA-ARMCTXM3-01** lab kit, which uses the **NXP LPC1768** microcontroller (ARM Cortex-M3). The **16x2 LCD** shows a road with two lanes. Our car stays near the left edge and obstacles come towards it from the right. The player uses an **analog joystick**: pushing it up or down changes lanes, pushing it left or right changes the speed, and pressing it down like a button starts the game, turns on a short shield, or pauses the game. The **four-digit seven-segment display** shows the remaining lives and the score.

- Three lives; a crash costs one life, and the game ends when all three are lost.
- Seven speed levels; the game automatically gets faster every 25 points.
- A shield that protects the car for a short time and then needs to recharge.
- A high score that is kept until the board is reset.

2. Why We Chose This Project

- **It combines everything from the lab in one program.** In the lab we used the LCD, seven-segment display, ADC and timers in separate experiments. A game needs all of them at the same time, which is much closer to how real embedded products work.
- **Mistakes are easy to see.** If the timing is wrong the display flickers, if input handling is wrong the car does not respond, and if the logic is wrong a crash is missed. This makes the game a good test of whether the design is correct.
- **It is cheap.** Everything except the joystick (about Rs 59) and a few jumper wires was already on the lab kit.
- **It is fun to demonstrate**, which makes the concepts easy to explain to anyone watching.

3. What We Used and Why

Table 1: Components and the reason for each

Component	Why we used it
LPC1768 kit (ALS-SDA-ARMCTXM3-01)	The lab kit we already know. The LPC1768 has the 12-bit ADC, timers and interrupts this game needs.
16x2 LCD (on the kit)	Shows the road. It lets us define our own 5x8 characters, which we used for the car, shielded car, obstacle and crash symbol.
Seven-segment display (on the kit)	Shows lives and score separately, so the LCD can be used fully for the road. It also shows display multiplexing.
Analog joystick module	One small part gives two analog axes and a button. The kit's own potentiometers are screwdriver trimmers, so they are not usable as a game controller.
10-core FRC cables (3)	The standard way to connect the kit's controller connectors (CNA-CND) to the on-board display blocks.
Female-to-female jumper wires (5)	Both sides have male pins: the joystick module has a male header, and the kit's CND header and 3.3 V test point are male pins. Female-to-female wires join male to male, need no soldering, and can be removed without changing the lab kit.
3.3 V supply for the joystick	The joystick output goes straight into the ADC, which only accepts 0-3.3 V. Powering it at 5 V could damage the pin.
Keil uVision4 and Flash Magic	The lab's standard tools for writing, compiling and downloading programs to this kit.

4. How Everything Is Connected

The kit brings the controller's pins out on four 10-pin connectors, CNA, CNB, CNC and CND. Pin 10 of every connector is ground. Each on-board display block has its own input connector, so we join them with ribbon cables, and we connect the joystick to the free pins of CND with jumper wires.

Table 2: Connections

From	To	Using	What it carries
CNC	CNAD (LCD)	FRC cable	LCD data D4-D7 (P0.15-P0.18), RS (P0.19), EN (P0.20); short JP16 and JP5
CNA	CNA2	FRC cable	Seven-segment segment lines a-g and dp (P0.4-P0.11)
CNB	CNB2	FRC cable	Seven-segment digit select (P1.23-P1.26)
Joystick VRx	CND pin 1	Jumper wire	Speed (left/right), P0.23 = ADC channel AD0.0
Joystick VRy	CND pin 2	Jumper wire	Lane (up/down), P0.24 = ADC channel AD0.1
Joystick SW	CND pin 7	Jumper wire	Button, P2.0
Joystick GND	CND pin 10	Jumper wire	Ground
Joystick +5V pin	3.3 V test point	Jumper wire	Power (3.3 V only)

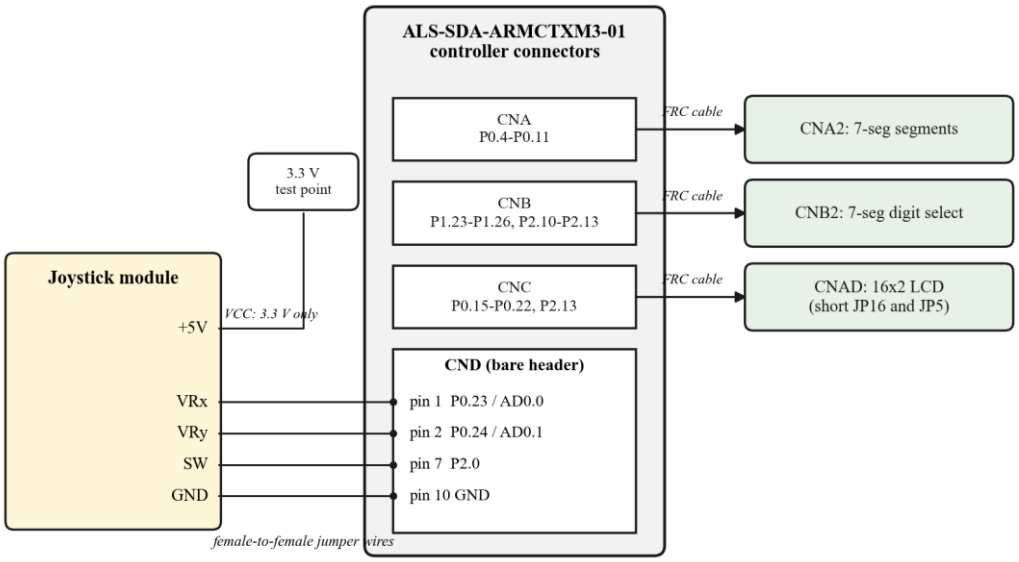


Figure 1: Connection diagram

Why the LCD is on CNC and not on CND. The lab manual connects the LCD through CND. But CND's first four pins are also the only analog inputs on the kit that are free to use. The other analog pins are taken: AD0.5 is permanently wired to a trimmer, AD0.4 is tied to another trimmer, and AD0.6 and AD0.7 are used for programming. CNC has the same pin order the LCD needs (four data lines, RS and EN), so moving the LCD there freed CND for both joystick axes. This was the most important design decision in the project.

5. How It Works

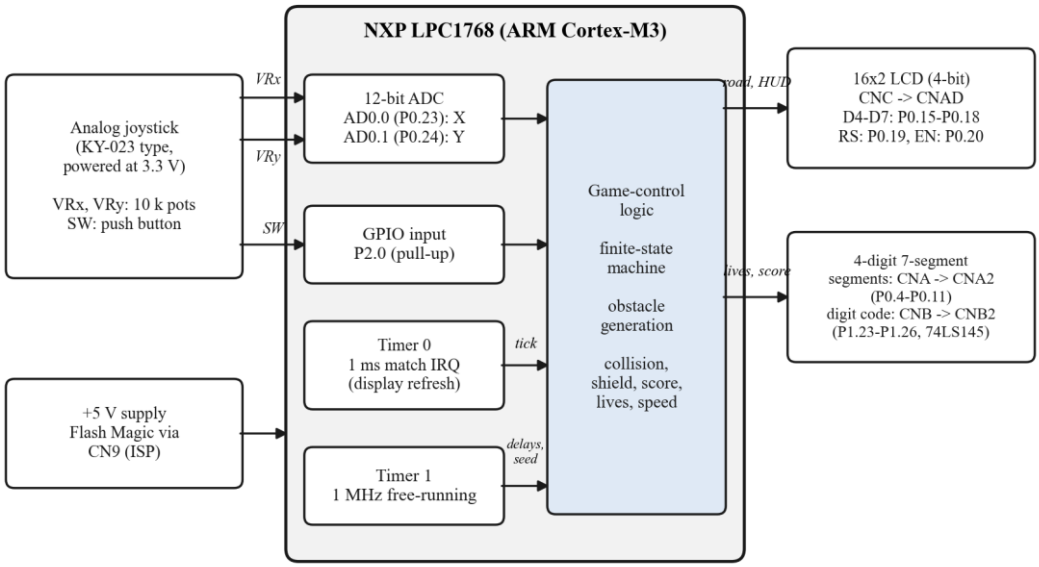


Figure 2: Block diagram

The program is written in Embedded C and is built around two ideas:

- **A 1 ms timer interrupt.** Timer 0 interrupts the processor every millisecond. Each time, it lights the next digit of the seven-segment display (one digit at a time, so fast that all four look steady) and counts time for the game.
- **A state machine instead of delay loops.** The game is always in one of six states: Start, Countdown, Playing, Paused, Collision or Game Over. Every 10 ms the main loop reads the joystick and button and does only the work of the current state, so nothing ever waits and the controls always respond.

While playing, the road moves one column to the left at a rate set by the speed level (from every 450 ms at level 1 to every 140 ms at level 7). New obstacles appear at the right edge using a random-number generator, with a rule that never blocks both lanes, so the player can always escape. If an obstacle reaches the car's position the game checks the shield: with the shield on, the obstacle is destroyed; without it, a life is lost.

Reading the joystick. The ADC turns each axis into a number from 0 to 4095; the resting position is about 2048. At power-on the program measures the real resting value (so do not touch the stick while the LCD says "Calibrating"). A push only counts when it moves more than 700 steps from the centre. This *dead zone* stops small shakes or electrical noise from moving the car. The button is checked three times, 10 ms apart, before a press is accepted (*debouncing*), and the program tells a short press from a long press (0.8 s).

Table 3: Controls

Action	Result
Push joystick up / down	Car moves to the upper / lower lane and stays there
Hold joystick right / left	Speed goes up / down one level every 0.3 s
Short press (start or game-over screen)	Starts a new game with a 3-2-1 countdown
Short press (while playing)	Turns on the shield if it is ready
Hold the button for 0.8 s	Pauses or resumes the game

What the displays show. LCD column 1 shows the speed level (top) and shield state (bottom: S = ready, * = active, . = recharging); the other 15 columns are the road. On the seven-segment display, the first digit is the number of lives (its dot lights when the shield is ready) and the last three digits are the score.

6. Problems We Faced and How We Solved Them

- **The joystick's X-axis did not work on the first pin we chose.** It was on AD0.5, which turned out to be permanently wired to a trimmer on the board. We solved it by moving the LCD to CNC and using the free analog pins on CND.
- **Running out of connectors.** The keypad and the seven-segment display both normally need connector CNB, so they cannot both be used with the standard wiring. The joystick replaced the keypad and needs no ribbon cable at all.

- **Two parts of the program writing to the same port.** The display refresh (in the interrupt) and the LCD code both use Port 0. We only use the FIOSET and FIOCLR registers, which change just the pins we name, so one part never overwrites the other's pins.
- **Choosing the button pin.** Pins P0.27 and P0.28 have no internal pull-up resistor, so the button would not read reliably there. We used P2.0, which has one.
- **A fair game.** Random obstacles could block both lanes. The code places at most one obstacle per column and keeps a gap between obstacles, so there is always a way through.

7. How the Project Can Be Scaled

- **Two players:** CND pins 3 and 4 (AD0.2, AD0.3) are now free, so a second joystick can be added without new hardware on the kit.
- **Better graphics:** the kit has an interface for a 128x64 graphic LCD, which would allow more lanes and smoother movement.
- **Saving the high score:** the LPC1768's real-time clock has battery-backed registers that keep data through a reset.
- **Scoreboard on a PC:** scores can be sent to a computer over the UART serial port.
- **More game features:** more obstacle types, levels and power-ups only need changes in the game logic, because the input, display and timing code are kept separate.
- **Faster input handling:** the ADC can be read using interrupts instead of waiting for each conversion.

8. Questions We May Be Asked

Q. Why did you use a joystick instead of the keypad?

A. It gives smooth, natural steering and speed control with one part, and it needs only jumper wires, while the keypad would use connector CNB that the seven-segment display already uses.

Q. Why female-to-female jumper wires?

A. The joystick has male header pins and the kit's CND header and 3.3 V point are also male pins. Female-to-female wires connect male to male without soldering and can be removed, so the lab kit is not modified.

Q. Why is the joystick powered from 3.3 V and not 5 V?

A. Its outputs go directly to the ADC pins, which accept only 0-3.3 V. At 5 V the output could exceed this limit and damage the microcontroller.

Q. How does the seven-segment display show four different digits?

A. By multiplexing: every millisecond the timer interrupt turns on just one digit with its own pattern. Each digit is refreshed 250 times a second, so all four appear to be on together.

Q. Why not use delay loops?

A. A delay loop stops the processor. The display would flicker and the controls would not respond while it waits. With a timer interrupt and a state machine, the program never waits.

Q. What is a dead zone and why is it needed?

A. A range around the joystick's centre where movement is ignored. Without it, noise or a small shake would change lanes on its own.

Q. How are collisions detected?

A. The road is stored as a 2 x 16 array. After every step, or whenever the car changes lane, the program checks whether the car's cell contains an obstacle.

Q. How are obstacles made random?

A. With a xorshift random-number generator, seeded from a running timer at the moment the player presses start, so every game is different.